

Recursion

The Pattern

- when a method calls itself
- Classic example--the factorial function:
 - $n! = 1 \cdot 2 \cdot 3 \cdot \dots \cdot (n-1) \cdot n$
- Recursive definition:

$$f(n) = \begin{cases} 1 & \text{if } n=0 \\ n \cdot f(n-1) & \text{else} \end{cases}$$

As a Python method:

```
1 def factorial(n):
2     if n == 0:
3         return 1
4     else:
5         return n * factorial(n-1)
```

Content of a Recursive Method

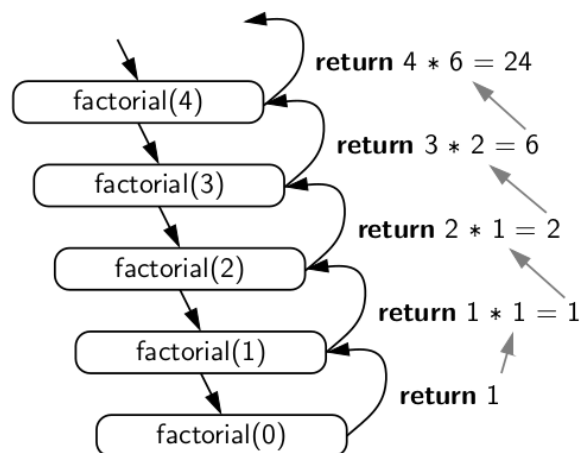
- **Base case(s)**
 - Values of the input variables for which we perform no recursive calls are called **base cases** (there should be at least one base case).
 - Every possible chain of recursive calls **must** eventually reach a base case.
- **Recursive calls**
 - Calls to the current method.
 - Each recursive call should be defined so that it makes progress towards a base case.

3

Visualizing

- **trace**
- **Example**

- A box for each recursive call
- An arrow from each caller to callee
- An arrow from each callee to caller showing return value



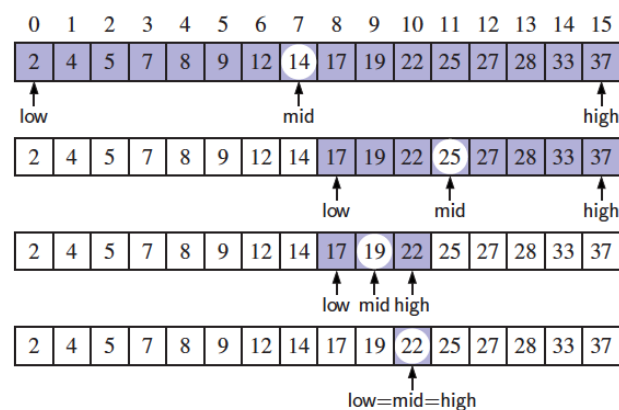
4

Recursive binary search

```
def bin_search(arr, key, low=0, high=None):  
    if high is None:  
        high = len(arr) - 1  
  
    if low > high:  
        return -1  
  
    mid = low + (high - low) // 2  
  
    if arr[mid] == key:  
        return mid  
    elif key < arr[mid]:  
        return bin_search(arr, key, low, mid - 1)  
    else:  
        return bin_search(arr, key, mid + 1, high)  
  
arr = [12, 51, 14, 123, 80, 78, 90, 178]  
arr.sort()  
x = 14  
  
status = bin_search(arr, x)
```

Visualizing Binary Search

- We consider three cases:
 - If the target equals data[mid], then we have found the target.
 - If target < data[mid], then we recur on the first half of the sequence.
 - If target > data[mid], then we recur on the second half of the sequence.



Computing Powers

- The power function, $p(x,n)=x^n$, can be defined recursively:

$$p(x,n)=\begin{cases} 1 & \text{if } n=0 \\ x \cdot p(x,n-1) & \text{else} \end{cases}$$

- This leads to a power function that runs in $O(n)$ time (for we make n recursive calls).
- We can do better than this, however.

7

Recursive Squaring

- We can derive a more efficient linearly recursive algorithm by using repeated squaring:

$$p(x,n)=\begin{cases} 1 & \text{if } x=0 \\ x \cdot p(x,(n-1)/2)^2 & \text{if } x>0 \text{ is odd} \\ p(x,n/2)^2 & \text{if } x>0 \text{ is even} \end{cases}$$

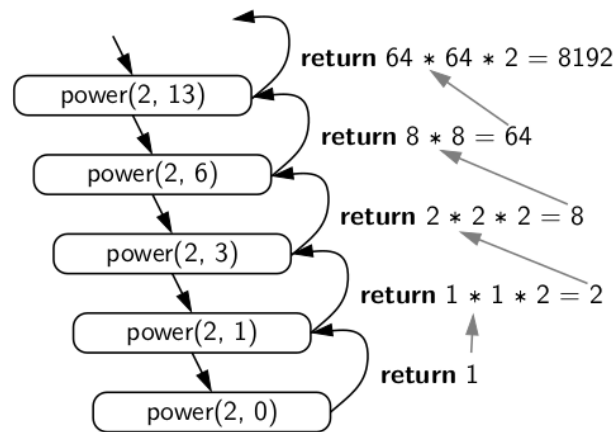
Time & Space complexities are $\log(n)$

8

Recursive Squaring Method

```
1 def power(x, n):
2     """ Compute the value x**n
3     if n == 0:
4         return 1
5     else:
6         partial = power(x, n // 2)
7         result = partial * partial
8         if n % 2 == 1:
9             result *= x
10        return result
```

Time & Space complexities are $\log(n)$



Reversing an Array

Algorithm ReverseArray(A , i , j):

Input: An array A and nonnegative integer indices i and j

Output: The reversal of the elements in A starting at index i and ending at j

Defining Arguments for

- In creating recursive methods, it is important to define the methods in ways that facilitate .
- For example, we defined the array reversal method as `ReverseArray(A, i, j)`, not `ReverseArray(A)`.

Python version:

```
1 def reverse(S, start, stop):
2     """Reverse elements in implicit slice S[start:stop]."""
3     if start < stop - 1:                # if at least 2 elements:
4         S[start], S[stop-1] = S[stop-1], S[start]    # swap first and last
5         reverse(S, start+1, stop-1)                # recur on rest
```

11

Tail Recursion

- Tail occurs when a linearly recursive method makes its recursive call as its last step. Eg. array reversal seen earlier.
- Such methods can be easily converted to non-recursive methods by compilers/interpreters (which saves on some resources).

Algorithm `IterativeReverseArray(A, i, j)`:

Input: An array *A* and non-negative integer indices *i* and *j*

Output: The reversal of the elements in *A* starting at index *i* and ending at *j*

while *i* < *j* **do**

 Swap *A*[*i*] and *A*[*j*]

i = *i* + 1

j = *j* - 1

return

12

Linear Recursion vs Binary Recursion

```
1 def linear_sum(S, n):
2     """ Return the sum of the first n numbers
3     if n == 0:
4         return 0
5     else:
6         return linear_sum(S, n-1) + S[n-1]
```

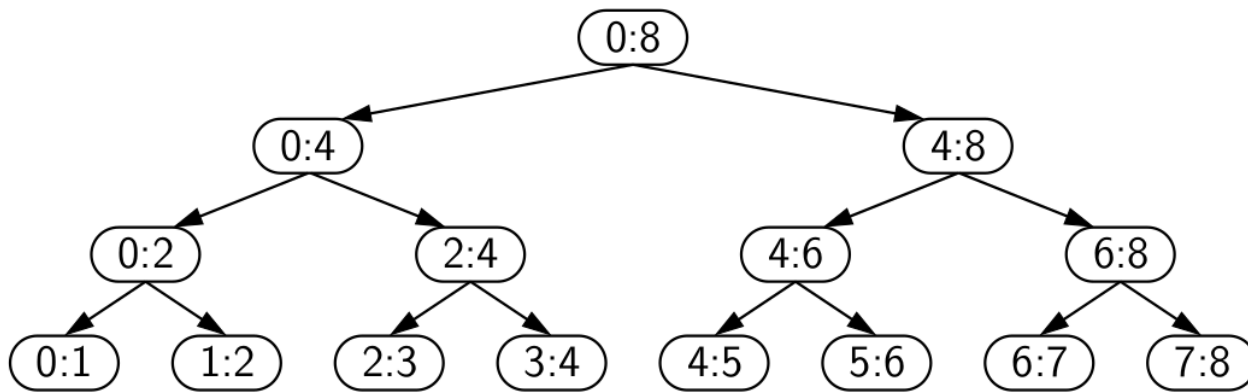
Linear Recursion vs Binary Recursion

```
1 def binary_sum(S, start, stop):
2     """ Return the sum of the numbers in implicit slice S[start:stop]. """
3     if start >= stop:                                # zero elements in slice
4         return 0
5     elif start == stop-1:                             # one element in slice
6         return S[start]
7     else:                                              # two or more elements in slice
8         mid = (start + stop) // 2
9         return binary_sum(S, start, mid) + binary_sum(S, mid, stop)
```

Note: There also exists multiple recursion

Recursion Trace

`binary_sum(A,0,8)` method



depth of the recursion is $1 + \log_2 n$

Time complexity is $O(n)$ (actually $2n-1$)

Space complexity is $O(\log(n))$

15

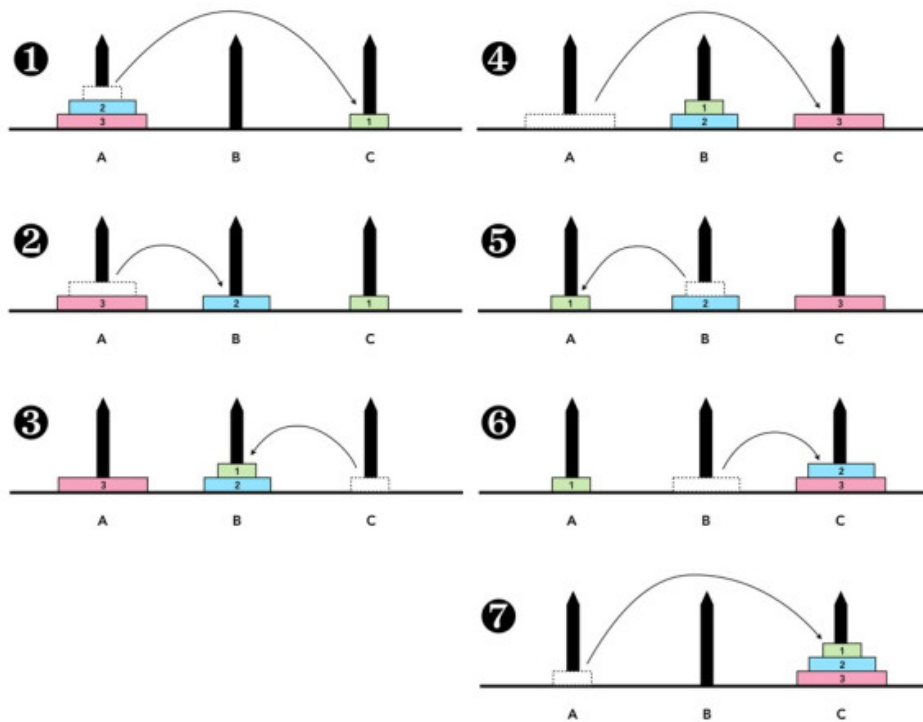
Tower of Hanoi



Procedure for moving a tower of n disks from a peg A onto a peg C, with B serving as an auxiliary peg:

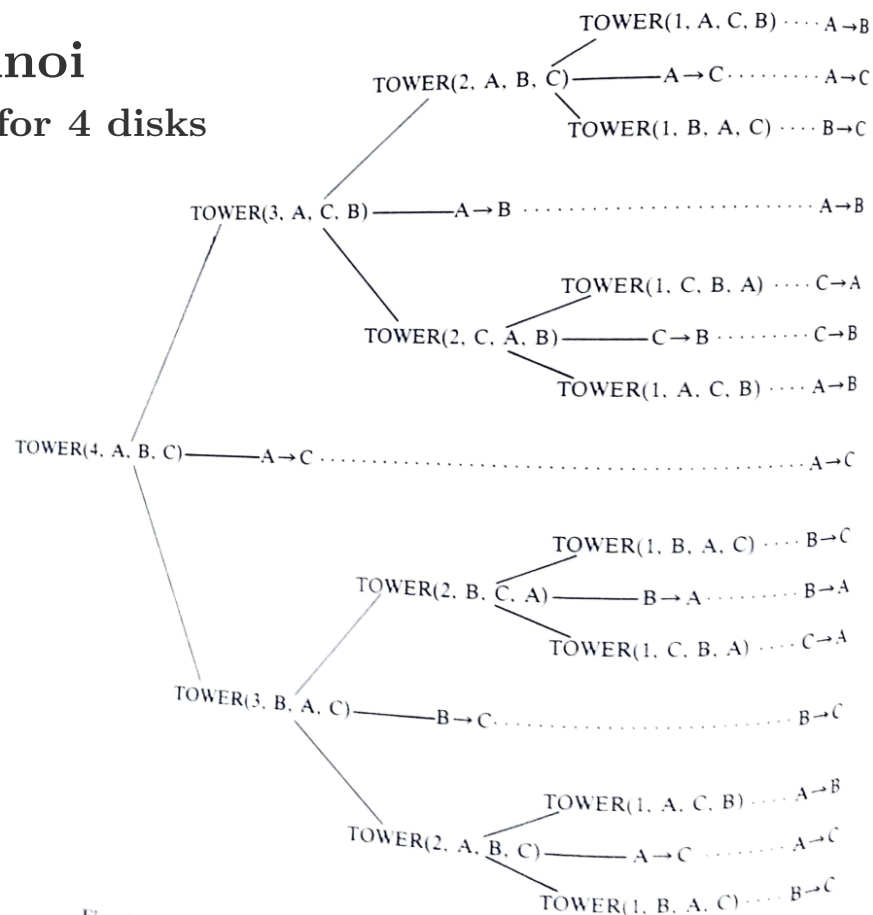
- 1) If $n > 1$, then first use this procedure to move the $n - 1$ smaller disks from peg A to peg B.
- 2) Now the largest disk, i.e. disk n can be moved from peg A to peg C.
- 3) If $n > 1$, then again use this procedure to move the $n - 1$ smaller disks from peg B to peg C.

Tower of Hanoi for 3 disks



Tower of Hanoi

Recursion trace for 4 disks



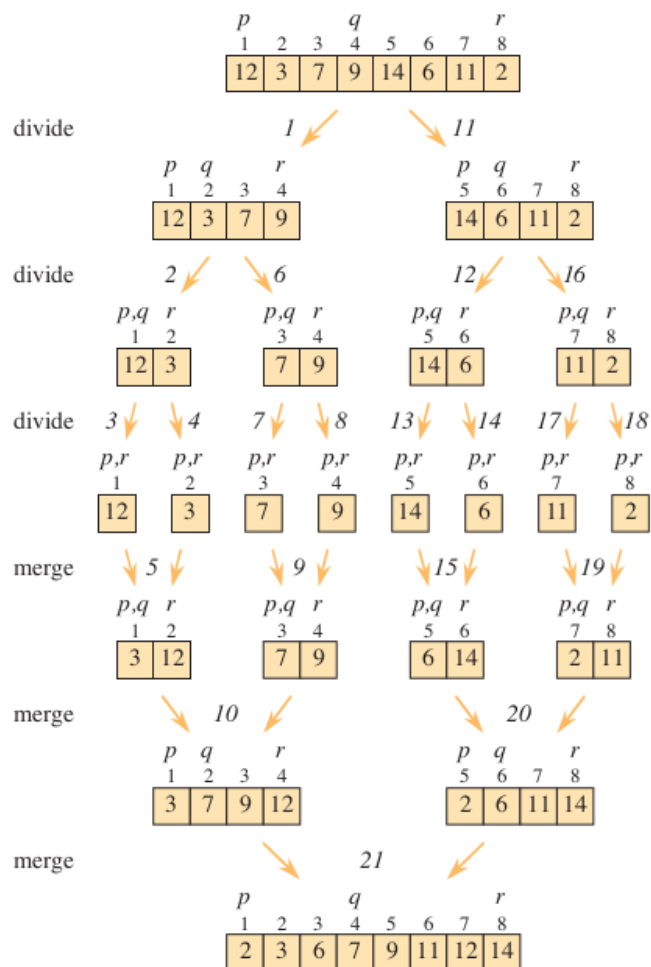
Divide & Conquer methods

- **Divide** the problem into one or more subproblems that are smaller instances of the same problem.
- **Conquer** the subproblems by solving them recursively.
- **Combine** the subproblem solutions to form a solution to the original problem.

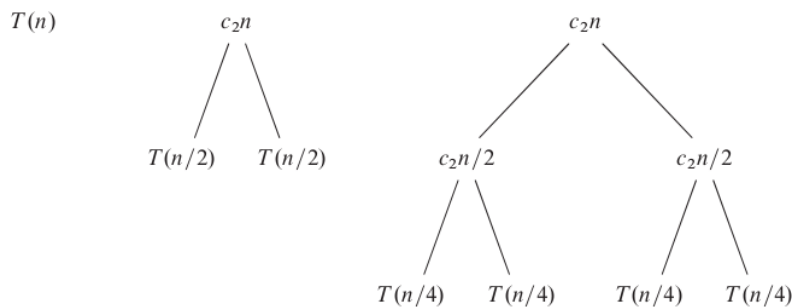
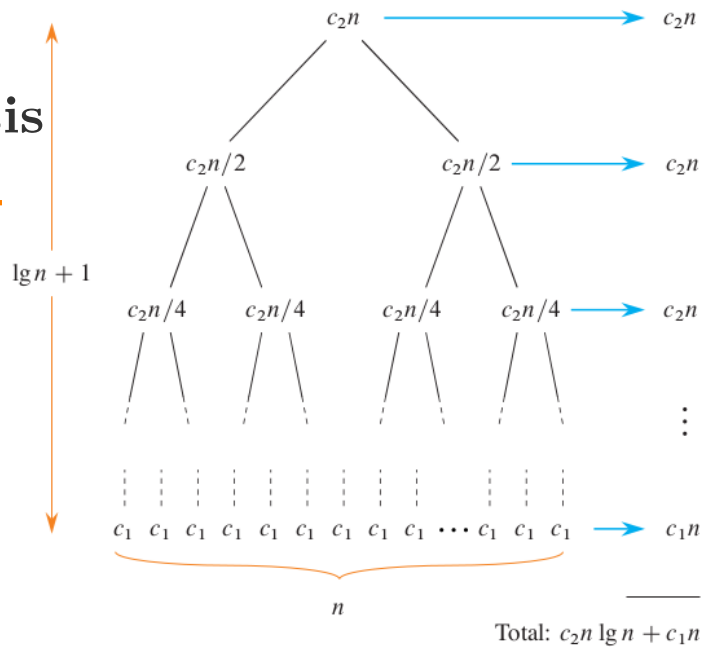
Merge Sort Analysis

$$T(n) = \begin{cases} \Theta(1) & \text{if } n < n_0, \\ D(n) + aT(n/b) + C(n) & \text{otherwise.} \end{cases}$$

$$T(n) = \begin{cases} c_1 & \text{if } n = 1, \\ 2T(n/2) + c_2n & \text{if } n > 1, \end{cases}$$



Merge Sort Analysis



Iterative Substitution

In the iterative substitution, or “plug-and-chug,” technique, we iteratively apply the recurrence equation to itself and see if we can find a pattern:

$$T(n) = 2T(n/2) + c_2n$$

$$2(2T(n/2^2)) + c_2(n/2) + c_2n$$

$$T(n) = 2^2T(n/2^2) + 2c_2n$$

$$T(n) = 2^3T(n/2^3) + 3c_2n$$

$$T(n) = 2^4T(n/2^4) + 4c_2n$$

...

$$T(n) = 2^iT(n/2^i) + ic_2n$$

Note that base, $T(1) = c_1$, case occurs when $2^i = n$. That is, $i = \log n$. So, $T(n) = c_1n + c_2n \log n$. Thus, $T(n)$ is $O(n \log n)$.

The Master Theorem

The Master Theorem applies to recurrences of the following form:

$$T(n) = aT(n/b) + f(n)$$

where $a \geq 1$ and $b > 1$ are constants and $f(n)$ is an asymptotically positive function.

1. If there exists a constant $\epsilon > 0$ such that $f(n) = O(n^{\log_b a - \epsilon})$, then $T(n) = \Theta(n^{\log_b a})$.
2. If there exists a constant $k \geq 0$ such that $f(n) = \Theta(n^{\log_b a} \lg^k n)$, then $T(n) = \Theta(n^{\log_b a} \lg^{k+1} n)$.
3. If there exists a constant $\epsilon > 0$ such that $f(n) = \Omega(n^{\log_b a + \epsilon})$, and if $f(n)$ additionally satisfies the **regularity condition** $af(n/b) \leq cf(n)$ for some constant $c < 1$ and all sufficiently large n , then $T(n) = \Theta(f(n))$. ■

23

Master Method, Example 1

1. If there exists a constant $\epsilon > 0$ such that $f(n) = O(n^{\log_b a - \epsilon})$, then $T(n) = \Theta(n^{\log_b a})$.
2. If there exists a constant $k \geq 0$ such that $f(n) = \Theta(n^{\log_b a} \lg^k n)$, then $T(n) = \Theta(n^{\log_b a} \lg^{k+1} n)$.
3. If there exists a constant $\epsilon > 0$ such that $f(n) = \Omega(n^{\log_b a + \epsilon})$, and if $f(n)$ additionally satisfies the **regularity condition** $af(n/b) \leq cf(n)$ for some constant $c < 1$ and all sufficiently large n , then $T(n) = \Theta(f(n))$. ■

$$T(n) = 4T(n/2) + n$$

Example:

Solution: $\log_b a = 2$, so case 1 says $T(n)$ is $\Theta(n^2)$.

24

Master Method, Example 2

1. If there exists a constant $\epsilon > 0$ such that $f(n) = O(n^{\log_b a - \epsilon})$, then $T(n) = \Theta(n^{\log_b a})$.
2. If there exists a constant $k \geq 0$ such that $f(n) = \Theta(n^{\log_b a} \lg^k n)$, then $T(n) = \Theta(n^{\log_b a} \lg^{k+1} n)$.
3. If there exists a constant $\epsilon > 0$ such that $f(n) = \Omega(n^{\log_b a + \epsilon})$, and if $f(n)$ additionally satisfies the **regularity condition** $af(n/b) \leq cf(n)$ for some constant $c < 1$ and all sufficiently large n , then $T(n) = \Theta(f(n))$. ■

Example:
$$T(n) = 2T(n/2) + n \log n$$

Solution: $\log_b a = 1$, so case 2 says $T(n)$ is $\Theta(n \log^2 n)$.

25

Master Method, Example 3

1. If there exists a constant $\epsilon > 0$ such that $f(n) = O(n^{\log_b a - \epsilon})$, then $T(n) = \Theta(n^{\log_b a})$.
2. If there exists a constant $k \geq 0$ such that $f(n) = \Theta(n^{\log_b a} \lg^k n)$, then $T(n) = \Theta(n^{\log_b a} \lg^{k+1} n)$.
3. If there exists a constant $\epsilon > 0$ such that $f(n) = \Omega(n^{\log_b a + \epsilon})$, and if $f(n)$ additionally satisfies the **regularity condition** $af(n/b) \leq cf(n)$ for some constant $c < 1$ and all sufficiently large n , then $T(n) = \Theta(f(n))$. ■

$$T(n) = T(n/3) + n \log n$$

Example:

Solution: $\log_b a = 0$, so case 3 says $T(n)$ is $\Theta(n \log n)$.

26

Master Method, Example 4

1. If there exists a constant $\epsilon > 0$ such that $f(n) = O(n^{\log_b a - \epsilon})$, then $T(n) = \Theta(n^{\log_b a})$.
2. If there exists a constant $k \geq 0$ such that $f(n) = \Theta(n^{\log_b a} \lg^k n)$, then $T(n) = \Theta(n^{\log_b a} \lg^{k+1} n)$.
3. If there exists a constant $\epsilon > 0$ such that $f(n) = \Omega(n^{\log_b a + \epsilon})$, and if $f(n)$ additionally satisfies the **regularity condition** $af(n/b) \leq cf(n)$ for some constant $c < 1$ and all sufficiently large n , then $T(n) = \Theta(f(n))$. ■

$$T(n) = 8T(n/2) + n^2$$

Example:

Solution: $\log_b a = 3$, so case 1 says $T(n)$ is $\Theta(n^3)$.

27

Master Method, Example 5

1. If there exists a constant $\epsilon > 0$ such that $f(n) = O(n^{\log_b a - \epsilon})$, then $T(n) = \Theta(n^{\log_b a})$.
2. If there exists a constant $k \geq 0$ such that $f(n) = \Theta(n^{\log_b a} \lg^k n)$, then $T(n) = \Theta(n^{\log_b a} \lg^{k+1} n)$.
3. If there exists a constant $\epsilon > 0$ such that $f(n) = \Omega(n^{\log_b a + \epsilon})$, and if $f(n)$ additionally satisfies the **regularity condition** $af(n/b) \leq cf(n)$ for some constant $c < 1$ and all sufficiently large n , then $T(n) = \Theta(f(n))$. ■

$$T(n) = 9T(n/3) + n^3$$

Example:

Solution: $\log_b a = 2$, so case 3 says $T(n)$ is $\Theta(n^3)$.

28

Master Method, Example 6

1. If there exists a constant $\epsilon > 0$ such that $f(n) = O(n^{\log_b a - \epsilon})$, then $T(n) = \Theta(n^{\log_b a})$.
2. If there exists a constant $k \geq 0$ such that $f(n) = \Theta(n^{\log_b a} \lg^k n)$, then $T(n) = \Theta(n^{\log_b a} \lg^{k+1} n)$.
3. If there exists a constant $\epsilon > 0$ such that $f(n) = \Omega(n^{\log_b a + \epsilon})$, and if $f(n)$ additionally satisfies the **regularity condition** $af(n/b) \leq cf(n)$ for some constant $c < 1$ and all sufficiently large n , then $T(n) = \Theta(f(n))$. ■

$$T(n) = T(n/2) + 1 \quad (\text{binary search})$$

Example:

Solution: $\log_b a = 0$, so case 2 says $T(n)$ is $\Theta(\log n)$.

29

Master Method, Example 7

1. If there exists a constant $\epsilon > 0$ such that $f(n) = O(n^{\log_b a - \epsilon})$, then $T(n) = \Theta(n^{\log_b a})$.
2. If there exists a constant $k \geq 0$ such that $f(n) = \Theta(n^{\log_b a} \lg^k n)$, then $T(n) = \Theta(n^{\log_b a} \lg^{k+1} n)$.
3. If there exists a constant $\epsilon > 0$ such that $f(n) = \Omega(n^{\log_b a + \epsilon})$, and if $f(n)$ additionally satisfies the **regularity condition** $af(n/b) \leq cf(n)$ for some constant $c < 1$ and all sufficiently large n , then $T(n) = \Theta(f(n))$. ■

$$T(n) = 2T(n/2) + \log n \quad (\text{heap construction})$$

Example:

Solution: $\log_b a = 1$, so case 1 says $T(n)$ is $\Theta(n)$.

30

When can the Master Theorem fail?

Will be discussed in the class

Computing Fibonacci Numbers

- Fibonacci numbers are defined recursively:

$$F_0 = 0$$

$$F_1 = 1$$

$$F_i = F_{i-1} + F_{i-2} \quad \text{for } i > 1.$$

- Recursive algorithm (first attempt):

Algorithm BinaryFib(k):

Input: Nonnegative integer k

Output: The k th Fibonacci number F_k

if $k \leq 1$ **then**

return k

else

return BinaryFib($k-1$) + BinaryFib($k-2$)

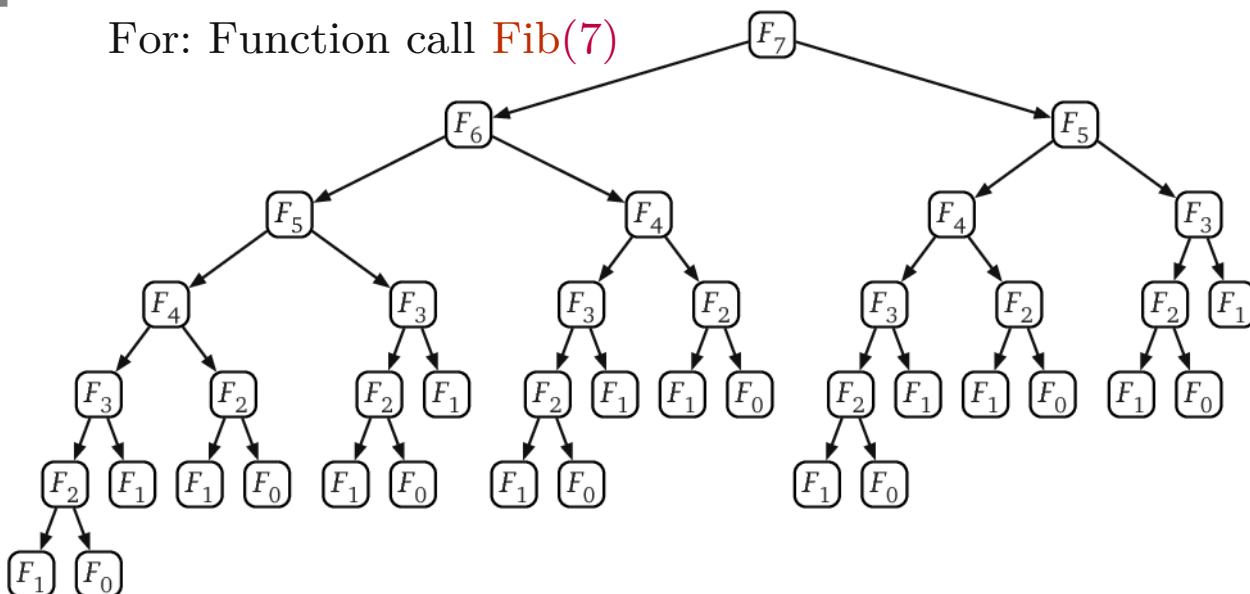
Analysis

- Let n_k be the number of recursive calls by **BinaryFib**(k)
 - $n_0 = 1$ $n_1 = 1$
 - $n_2 = n_1 + n_0 = 2$
 - $n_3 = n_2 + n_1 = 3$
 - $n_4 = n_3 + n_2 = 5$
 - $n_5 = n_4 + n_3 = 8$
 - $n_6 = n_5 + n_4 = 13$
 - $n_7 = n_6 + n_5 = 21$
- Note that n_k at least doubles every other time
- That is, $n_k > 2^{k/2}$. It is exponential!

33

Analysis

For: Function call **Fib**(7)



Can we do better?

34

Integer Multiplication

Algorithm: Multiply two n-digit integers I and J.

Divide step: Split I and J into high-order and low-order halves

$$\begin{array}{l} X = r^{n/2} A + B \\ Y = r^{n/2} C + D \end{array} \quad \begin{array}{|c|c|} \hline A & B \\ \hline C & D \\ \hline \end{array}$$

We can then define $X*Y$ by multiplying the parts and adding:

$$XY = \underbrace{AC}_{Z_2} r^n + \underbrace{(AD + BC)}_{Z_1} r^{n/2} + \underbrace{BD}_{Z_0}$$

So, $T(n) = 4T(n/2) + O(n)$, which implies $T(n)$ is $O(n^2)$.

But that is no better than the algorithm we learned in grade ³⁵

An Improved Integer Multiplication Algorithm: Karatsuba Algorithm

Algorithm: Multiply two n-digit integers I and J.

Divide step: Split I and J into high-order and low-order halves

$$\begin{array}{l} X = r^{n/2} A + B \\ Y = r^{n/2} C + D \end{array} \quad \begin{array}{|c|c|} \hline A & B \\ \hline C & D \\ \hline \end{array}$$

Observe that there is a different way to multiply parts:

$$\begin{array}{l} Z_0 = BD \\ Z_2 = AC \\ Z_1 = (A+B)(C+D) - Z_0 - Z_2 \end{array} \quad XY = \underbrace{AC}_{Z_2} r^n + \underbrace{(AD + BC)}_{Z_1} r^{n/2} + \underbrace{BD}_{Z_0}$$

So, $T(n) = 3T(n/2) + O(n)$, which implies $T(n)$ is $O(n^{\log_2 3}) = O(n^{1.584})$, by the Master Theorem. Thus, $T(n)$ is $O(n^{1.585})$.

References

Data Structures and Algorithms in Python

Michael T. Goodrich, Roberto Tamassia, Michael H. Goldwasser

Introduction to Algorithms

Leiserson, Stein, Rivest, Cormen

Algorithms, 4th Edition

Robert Sedgewick and Kevin Wayne

Few Images from the internet